

MYSQL STORED PROCEDURES

Examples and Practice Exercises for MySQL Workbench

2520030396

Putti mokshitha

1. Setup: Employees Table

```
CREATE DATABASE IF NOT EXISTS employeeedb1;
```

```
USE employeeedb1;
```

```
CREATE TABLE IF NOT EXISTS employees (
```

```
    emp_id INT PRIMARY KEY,
```

```
    emp_name VARCHAR(50),
```

```
    dept_id INT,
```

```
    salary DECIMAL(10,2)
```

```
);
```

```
INSERT INTO employees VALUES
```

```
(101,'Rahul',1,50000),
```

```
(102,'Priya',2,65000),
```

```
(103,'Anil',1,55000),
```

```
(104,'Sneha',3,70000),
```

```
(105,'Kiran',2,48000);
```

The screenshot shows the MySQL Workbench interface with the following components:

- Navigator:** Shows the 'Local instance MySQL80' connection. The 'SCHEMAS' tab is selected, showing a list of schemas including 'employeeedb1'.
- Query Editor:** Contains the following SQL code:

```
1 CREATE DATABASE IF NOT EXISTS employeeedb1;
2
3 USE employeeedb1;
4
5 CREATE TABLE IF NOT EXISTS employees (
6     emp_id INT PRIMARY KEY,
7     emp_name VARCHAR(50),
8     dept_id INT,
9     salary DECIMAL(10,2)
10 );
11
12 INSERT INTO employees VALUES
13 (101,'Rahul',1,50000),
14 (102,'Priya',2,65000),
15 (103,'Anil',1,55000),
16 (104,'Sneha',3,70000),
17 (105,'Kiran',2,48000);
```
- SQL Additions:** A message box stating: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."
- Output:** A table showing the execution results of the queries:

#	Time	Action	Message	Duration / Fetch
1	08:08:58	CREATE DATABASE IF NOT EXISTS employeeedb1	1 row(s) affected	0.016 sec
2	08:08:58	USE employeeedb1	0 row(s) affected	0.000 sec
3	08:08:58	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, emp_name VARCHAR(50), d...	0 row(s) affected	0.031 sec
4	08:08:58	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), (104,'Sneha'...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.016 sec

2. Verify Data

Question: Display all employee records.

```
SELECT * FROM employees;
```

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session

Query 1 x

```

12 INSERT INTO employees VALUES
13 (101,'Rahul',1,50000),
14 (102,'Priya',2,65000),
15 (103,'Anil',1,55000),
16 (104,'Sneha',3,70000),
17 (105,'Kiran',2,48000);
18 SELECT * FROM employees;

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	50000.00
102	Priya	2	65000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00

employees 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
4	08:08:58	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), (104,'Sne...	5 row(s) affected. Records: 5 Duplicates: 0 Warnings: 0	0.016 sec
5	08:17:23	CREATE DATABASE IF NOT EXISTS employeesdb1	1 row(s) affected, 1 warning(s): 1007 Can't create database 'employeesdb1': database exists	0.000 sec
6	08:17:23	USE employeesdb1	0 row(s) affected	0.000 sec
7	08:17:23	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, emp_name VARCHAR(50), ...	0 row(s) affected, 1 warning(s): 1050 Table 'employees' already exists	0.000 sec
8	08:17:23	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), (104,'Sne...	Error Code: 1062 Duplicate entry '101' for key 'employees.PRIMARY'	0.000 sec
9	08:21:11	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Procedure 1 – Display All Employees

Question: Create a procedure that displays all employees.

DROP PROCEDURE IF EXISTS GetEmployees;

DELIMITER \$\$

CREATE PROCEDURE GetEmployees()

BEGIN

 SELECT * FROM employees;

END \$\$

DELIMITER ;

CALL GetEmployees();

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session

Query 1 x

```

6 BEGIN
7     SELECT * FROM employees;
8 END $$
9
10 DELIMITER ;
11
12 CALL GetEmployees();

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	50000.00
102	Priya	2	65000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00

Result 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
12	08:24:12	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, emp_name VARCHAR(50), ...	0 row(s) affected, 1 warning(s): 1050 Table 'employees' already exists	0.016 sec
13	08:24:12	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), (104,'Sne...	Error Code: 1062 Duplicate entry '101' for key 'employees.PRIMARY'	0.000 sec
14	08:24:34	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected, 1 warning(s): 1305 PROCEDURE employeesdb1.GetEmployees does not exist	0.016 sec
15	08:26:18	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected, 1 warning(s): 1305 PROCEDURE employeesdb1.GetEmployees does not exist	0.016 sec
16	08:26:18	CREATE PROCEDURE GetEmployees() BEGIN SELECT * FROM employees; END	0 row(s) affected	0.000 sec
17	08:26:18	CALL GetEmployees()	5 row(s) returned	0.000 sec / 0.000 sec

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Procedure 2 – IN Parameter

Question: Create a procedure that accepts an employee ID and displays that employee's details.

```
DROP PROCEDURE IF EXISTS GetEmployeeById;  
DELIMITER $$  
CREATE PROCEDURE GetEmployeeById(IN eid INT)  
BEGIN  
    SELECT * FROM employees  
    WHERE emp_id = eid;  
END $$  
DELIMITER ;  
CALL GetEmployeeById(102);
```

The screenshot shows the MySQL Workbench interface. The 'Query 1' tab is active, displaying the SQL code for the stored procedure. The 'Result Grid' shows the output of the procedure call, which is a single row for employee ID 102, named Priya, in department 2, with a salary of 65000.00. The 'Output' tab at the bottom shows the execution log, including the creation of the procedure, the call, and the resulting output.

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup

Query 1

```
9 SELECT * FROM employees  
10 WHERE emp_id = eid;  
11 END $$  
12  
13 DELIMITER ;  
14  
15 CALL GetEmployeeById(102);
```

Result Grid

emp_id	emp_name	dept_id	salary
102	Priya	2	65000.00

SQLAdditions

Automatic context help is disabled. Use the toolbar to toggle automatic context help on or off.

Administration Schemas

Information

No object selected

Output

Action Output

#	Time	Action	Message
✓ 16	08:26:18	CREATE PROCEDURE GetEmployeeById() BEGIN SELECT * FROM employees; END	0 row(s) affected
✓ 17	08:26:18	CALL GetEmployeeById()	5 row(s) returned
✓ 18	08:30:45	USE employee_db1	0 row(s) affected
✓ 19	08:30:45	DROP PROCEDURE IF EXISTS GetEmployeeById	0 row(s) affected, 1 warning(s): 1305 PROCEDURE employee_db1.GetEmployeeById does not exist
✓ 20	08:30:45	CREATE PROCEDURE GetEmployeeById(IN eid INT) BEGIN SELECT * FROM employees WHERE emp_id = eid; END	0 row(s) affected
✓ 21	08:30:45	CALL GetEmployeeById(102)	1 row(s) returned

Procedure 3 – Update Salary

Question: Create a procedure that accepts employee ID and amount, then increases the employee's salary.

```
DROP PROCEDURE IF EXISTS IncreaseSalary;  
DELIMITER $$  
CREATE PROCEDURE IncreaseSalary(  
    IN eid INT,  
    IN amount DECIMAL(10,2)  
)  
BEGIN  
    UPDATE employees
```

```

SET salary = salary + amount
WHERE emp_id = eid;
END $$
DELIMITER ;
CALL IncreaseSalary(101,5000);
SELECT * FROM employees WHERE emp_id=101;

```

The screenshot shows the MySQL Workbench interface for a local instance of MySQL 8.0. The 'Query 1' tab is active, displaying the following SQL code:

```

16
17 DELIMITER ;
18
19 • CALL IncreaseSalary(101,5000);
20
21 • SELECT * FROM employees
22 WHERE emp_id = 101;

```

The 'Result Grid' shows the output of the query, displaying a single row for employee 101, Rahul, with a salary of 55000.00. The grid also shows a row of NULL values for the other columns.

emp_id	emp_name	dept_id	salary
101	Rahul	1	55000.00
NULL	NULL	NULL	NULL

The interface also includes a sidebar with navigation options like 'Server Status', 'Client Connections', and 'Performance Reports'. The bottom status bar indicates 'No object selected'.

Procedure 4 – Aggregate Function

Question: Create a procedure to calculate the average salary of all employees.

```

DROP PROCEDURE IF EXISTS AvgSalary;
DELIMITER $$
CREATE PROCEDURE AvgSalary()
BEGIN
    SELECT AVG(salary) AS AverageSalary
    FROM employees;
END $$
DELIMITER ;
CALL AvgSalary();

```

The screenshot shows the MySQL Workbench interface for a local instance of MySQL 8.0. The 'Query' tab is active, displaying the following SQL code:

```

9      SELECT AVG(salary) AS AverageSalary
10     FROM employees;
11   END $$
12
13   DELIMITER ;
14
15 • CALL AvgSalary();

```

The 'Result Grid' shows the output of the query:

AverageSalary
58600.000000

The 'Output' tab at the bottom shows the execution log:

#	Time	Action	Message
25	08:31:46	CALL IncreaseSalary(101,5000)	1 row(s) affected
26	08:31:46	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
27	08:32:30	USE employeeedb1	0 row(s) affected
28	08:32:30	DROP PROCEDURE IF EXISTS AvgSalary	0 row(s) affected

Procedure 5 – JOIN

Question: Create a procedure to display employee name, department name and salary. Run the department setup first.

```

CREATE TABLE IF NOT EXISTS departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
INSERT IGNORE INTO departments VALUES
(1,'CSE'),(2,'ECE'),(3,'EEE');
DROP PROCEDURE IF EXISTS EmployeeDepartment;
DELIMITER $$
CREATE PROCEDURE EmployeeDepartment()
BEGIN
    SELECT e.emp_name, d.dept_name, e.salary
    FROM employees e
    JOIN departments d ON e.dept_id=d.dept_id;
END $$
DELIMITER ;
CALL EmployeeDepartment();

```

MySQL Workbench interface showing a query execution session.

Query 1:

```

20 FROM employees e
21 JOIN departments d ON e.dept_id = d.dept_id;
22 END $$
23
24 DELIMITER ;
25
26 CALL EmployeeDepartment();

```

Result Grid:

emp_name	dept_name	salary
Rahul	CSE	55000.00
Priya	ECE	65000.00
Anil	CSE	55000.00
Sneha	EEE	70000.00
Kiran	ECE	48000.00

Output:

#	Time	Action	Message
31	08:33:49	USE employeedb1	0 row(s) affected
32	08:33:49	CREATE TABLE IF NOT EXISTS departments (dept_id INT PRIMARY KEY, dept_name VARCHAR(50))	0 row(s) affected
33	08:33:49	INSERT IGNORE INTO departments VALUES (1,'CSE'), (2,'ECE'), (3,'EEE')	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
34	08:33:49	DROP PROCEDURE IF EXISTS EmployeeDepartment	0 row(s) affected, 1 warning(s): 1305 PROCEDURE employeedb1.EmployeeDep...
35	08:33:49	CREATE PROCEDURE EmployeeDepartment() BEGIN SELECT e.emp_name, d.dept_name, e.salary FR...	0 row(s) affected
36	08:33:49	CALL EmployeeDepartment()	5 row(s) returned

Procedure 6 – GROUP BY and AVG

Question: Create a procedure to display department-wise employee count and average salary.

DROP PROCEDURE IF EXISTS DepartmentAverageSalary;

DELIMITER \$\$

CREATE PROCEDURE DepartmentAverageSalary()

BEGIN

 SELECT d.dept_name,
 COUNT(e.emp_id) AS EmployeeCount,
 AVG(e.salary) AS AverageSalary

 FROM employees e

 JOIN departments d ON e.dept_id=d.dept_id

 GROUP BY d.dept_id, d.dept_name;

END \$\$

DELIMITER ;

CALL DepartmentAverageSalary();

The screenshot shows the MySQL Workbench interface for a local instance of MySQL 8.0. The main window displays a query in the 'Query 1' editor. The query is as follows:

```

13 JOIN departments d ON e.dept_id = d.dept_id
14 GROUP BY d.dept_id, d.dept_name;
15 END $$
16
17 DELIMITER ;
18
19 CALL DepartmentAverageSalary();

```

Below the query editor, the 'Result Grid' is visible, showing the results of the query. The grid has three columns: dept_name, EmployeeCount, and AverageSalary. The results are as follows:

dept_name	EmployeeCount	AverageSalary
CSE	2	55000.000000
ECE	2	56500.000000
EEE	1	70000.000000

On the right side of the interface, a message states: 'Automatic context help is disabled current caret position'. The bottom panel shows the 'Output' tab with a list of actions and their messages, including the execution of the query and the results of the procedure calls.

Procedure 7 – IF...ELSE

Question: Display High Salary if salary is at least 60000; otherwise display Low Salary.

DROP PROCEDURE IF EXISTS SalaryStatus;

DELIMITER \$\$

CREATE PROCEDURE SalaryStatus(IN eid INT)

BEGIN

DECLARE sal DECIMAL(10,2);

SELECT salary INTO sal

FROM employees

WHERE emp_id=eid;

IF sal >= 60000 THEN

SELECT 'High Salary' AS Status;

ELSE

SELECT 'Low Salary' AS Status;

END IF;

END \$\$

DELIMITER ;

CALL SalaryStatus(102);

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session

Query 1

```

1 • USE employeeedb1;
2
3 DROP PROCEDURE IF EXISTS SalaryStatus;
4
5 DELIMITER $$
6
7 CREATE PROCEDURE SalaryStatus(IN eid INT)
8 BEGIN

```

Result Grid

Status

High Salary

Result 9

Read Only

Context Help Snippets

Output

Action Output

#	Time	Action	Message
43	08:37:18	CREATE PROCEDURE DepartmentAverageSalary() BEGIN SELECT d.dept_name, COUNT(e.emp_i...	0 row(s) affected
44	08:37:18	CALL DepartmentAverageSalary()	3 row(s) returned
45	08:37:54	USE employeeedb1	0 row(s) affected
46	08:37:54	DROP PROCEDURE IF EXISTS SalaryStatus	0 row(s) affected
47	08:37:54	CREATE PROCEDURE SalaryStatus(IN eid INT) BEGIN DECLARE sal DECIMAL(10,2); SELECT salary ...	0 row(s) affected
48	08:37:54	CALL SalaryStatus(102)	1 row(s) returned

Procedure 8 – OUT Parameter

Question: Return the total number of employees using an OUT parameter.

DROP PROCEDURE IF EXISTS EmployeeCount;

DELIMITER \$\$

CREATE PROCEDURE EmployeeCount(OUT total INT)

BEGIN

SELECT COUNT(*) INTO total

FROM employees;

END \$\$

DELIMITER ;

CALL EmployeeCount(@total);

SELECT @total AS TotalEmployees;

mysql workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 x

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session

Query 1 x

Limit to 1000 rows

```

11 END $$
12
13 DELIMITER ;
14
15 CALL EmployeeCount(@total);
16
17 SELECT @total AS TotalEmployees;

```

Result Grid

TotalEmployees
5

Result 10 x

Read Only

Context Help Snippets

Output

Action Output

#	Time	Action	Message
48	08:37:54	CALL SalaryStatus(102)	1 row(s) returned
49	08:39:17	USE employeeedb1	0 row(s) affected
50	08:39:17	DROP PROCEDURE IF EXISTS EmployeeCount	0 row(s) affected, 1 warning(s): 1305 P
51	08:39:17	CREATE PROCEDURE EmployeeCount(OUT total INT) BEGIN SELECT COUNT(*) INTO total FROM em...	0 row(s) affected
52	08:39:17	CALL EmployeeCount(@total)	1 row(s) affected
53	08:39:17	SELECT @total AS TotalEmployees LIMIT 0, 1000	1 row(s) returned

Procedure 9 – INOUT Parameter

Question: Receive a bonus through an INOUT parameter and add 5000 to it.

DROP PROCEDURE IF EXISTS AddBonus;

DELIMITER \$\$

CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))

BEGIN

SET bonus = bonus + 5000;

END \$\$

DELIMITER ;

SET @bonus=10000;

CALL AddBonus(@bonus);

SELECT @bonus AS UpdatedBonus;

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session

Query 1

```

12 DELIMITER ;
13
14 • SET @bonus = 10000;
15
16 • CALL AddBonus(@bonus);
17
18 • SELECT @bonus AS UpdatedBonus;

```

Result Grid

UpdatedBonus
15000.00

Result 12

Output

Action Output

#	Time	Action	Message
59	08:41:11	USE employeeedb1	0 row(s) affected
60	08:41:11	DROP PROCEDURE IF EXISTS AddBonus	0 row(s) affected, 1
61	08:41:11	CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2)) BEGIN SET bonus = bonus + 5000; END	0 row(s) affected
62	08:41:11	SET @bonus = 10000	0 row(s) affected
63	08:41:11	CALL AddBonus(@bonus)	0 row(s) affected
64	08:41:11	SELECT @bonus AS UpdatedBonus LIMIT 0, 1000	1 row(s) returned

Procedure 10 – Transaction

Question: Transfer an amount from one employee salary record to another using START TRANSACTION and COMMIT.

DROP PROCEDURE IF EXISTS TransferSalaryAmount;

DELIMITER \$\$

CREATE PROCEDURE TransferSalaryAmount(

IN fromEmp INT,

IN toEmp INT,

IN amount DECIMAL(10,2)

)

BEGIN

START TRANSACTION;

UPDATE employees

SET salary=salary-amount

WHERE emp_id=fromEmp;

UPDATE employees

SET salary=salary+amount

```

WHERE emp_id=toEmp;
COMMIT;
END $$
DELIMITER ;
CALL TransferSalaryAmount(101,102,2000);
SELECT * FROM employees WHERE emp_id IN (101,102);

```

The screenshot shows the MySQL Workbench interface for a local instance of MySQL 8.0. The 'Query 1' tab is active, displaying the following SQL script:

```

25
26 DELIMITER ;
27
28 CALL TransferSalaryAmount(101,102,2000);
29
30 SELECT * FROM employees
31 WHERE emp_id IN (101,102);

```

The 'Result Grid' shows the output of the query, displaying two rows of employee data:

emp_id	emp_name	dept_id	salary
101	Rahul	1	53000.00
102	Priya	2	67000.00

The 'Output' tab at the bottom shows the execution log, including the following actions and messages:

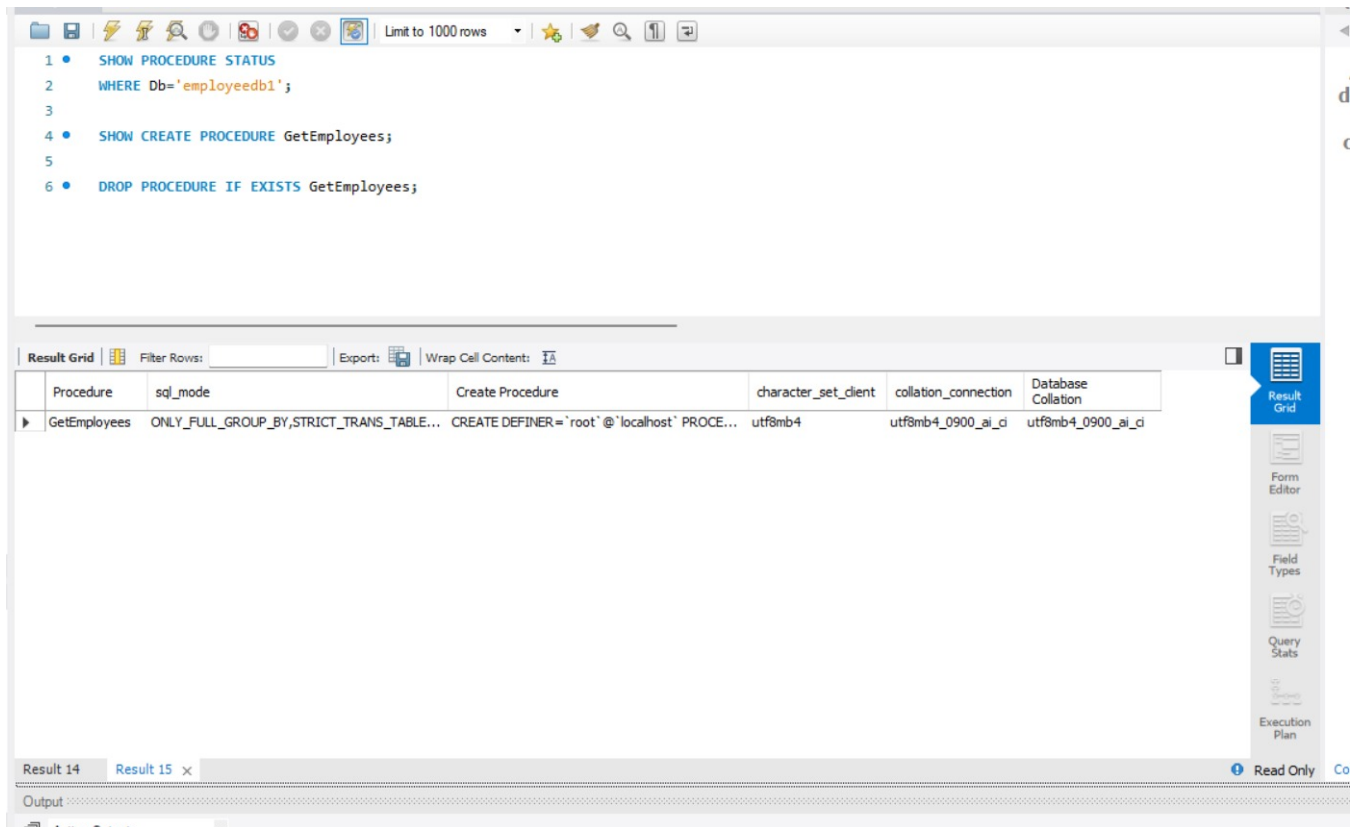
#	Time	Action	Message
64	08:41:11	SELECT @bonus AS UpdatedBonus LIMIT 0, 1000	1 row(s) returned
65	08:42:04	USE employeeedb1	0 row(s) affected
66	08:42:04	DROP PROCEDURE IF EXISTS TransferSalaryAmount	0 row(s) affected, 1 warning
67	08:42:04	CREATE PROCEDURE TransferSalaryAmount(IN fromEmp INT, IN toEmp INT, IN amount DECIMAL(...	0 row(s) affected
68	08:42:04	CALL TransferSalaryAmount(101,102,2000)	0 row(s) affected
69	08:42:04	SELECT * FROM employees WHERE emp_id IN (101,102) LIMIT 0, 1000	2 row(s) returned

4. Useful Procedure Commands

```

SHOW PROCEDURE STATUS
WHERE Db='employeeedb1';
SHOW CREATE PROCEDURE GetEmployees;
DROP PROCEDURE IF EXISTS GetEmployees;

```



5. Why DELIMITER is Used

DELIMITER temporarily changes the statement terminator in MySQL Workbench. It prevents the semicolons inside BEGIN...END from being interpreted as the end of the entire CREATE PROCEDURE statement.

DELIMITER \$\$

CREATE PROCEDURE Example()

BEGIN

 SELECT * FROM employees;

END \$\$

DELIMITER ;

6. Student Practice Tasks

1. Create a procedure to display employees whose salary is greater than a given amount.

Query 1

Search table data for text in objects selected in the sidebar schema tree

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS EmployeesAboveSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE EmployeesAboveSalary(IN min_salary INT)
6 BEGIN
7     SELECT *
8     FROM employees
9     WHERE salary > min_salary;
10 END $$

```

Result Grid

emp_id	emp_name	dept_id	salary
102	Priya	2	67000.00
104	Sneha	3	70000.00

Result Grid

Form Editor

Result 12

Read Only

Context Help

Output

Action Output

#	Time	Action	Message
42	12:11:28	CREATE PROCEDURE TransferSalaryAmount(...	0 row(s) affected
43	12:11:28	CALL TransferSalaryAmount(101,102,2000)	0 row(s) affected
44	12:11:28	SELECT * FROM employees WHERE emp_id IN (...	2 row(s) returned
45	12:12:52	DROP PROCEDURE IF EXISTS EmployeesAboveSalary	0 row(s) affected, 1 warning(s): 1305 PE

2. Create a procedure to update an employee's department using IN parameters.

Query 1 x

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS UpdateEmployeeDepartment
2
3 DELIMITER $$
4
5 CREATE PROCEDURE UpdateEmployeeDepartment(
6     IN eid INT,
7     IN new_dept INT
8 )
9 BEGIN
10     UPDATE employees

```

Result Grid

emp_id	emp_name	dept_id	salary
102	Priya	3	67000.00

employees 13 x

Apply Context Help

3. Create a procedure to return the maximum salary using an OUT parameter.

Query 1 x

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS MaxSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE MaxSalary(OUT max_sal DECIMAL(
6 BEGIN
7     SELECT MAX(salary) INTO max_sal
8     FROM employees;
9 END $$
10

```

Result Grid

MaximumSalary
70000.00

Result 14 x

Read Only Context Help

Output

Action Output

#	Time	Action	Message
50	12:14:58	CALL UpdateEmployeeDepartment(102,3)	1 row(s) affected
51	12:14:58	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
52	12:15:54	DROP PROCEDURE IF EXISTS MaxSalary	0 row(s) affected, 1 warning(s): 130

4. Create a procedure to return the employee count of a specified department.

The screenshot shows the SQL Developer interface with a query window titled 'Query 1'. The SQL code is as follows:

```

5 CREATE PROCEDURE DepartmentEmployeeCount(IN dept
6 BEGIN
7     SELECT COUNT(*) AS EmployeeCount
8     FROM employees
9     WHERE dept_id = dept;
10 END $$
11
12 DELIMITER ;
13
14 CALL DepartmentEmployeeCount(2);

```

The 'Result Grid' shows one row with the value '1' under the column 'EmployeeCount'.

The 'Output' window shows the following messages:

#	Time	Action	Message
56	12:16:28	DROP PROCEDURE IF EXISTS DepartmentEmpl...	0 row(s) affected, 1 warning(s): 1305
57	12:16:28	CREATE PROCEDURE DepartmentEmployeeCo...	0 row(s) affected
58	12:16:28	CALL DepartmentEmployeeCount(2)	1 row(s) returned

5. Create a procedure using JOIN to display employee and department details.

The screenshot shows the SQL Developer interface with a query window titled 'Query 1'. The SQL code is as follows:

```

1 DROP PROCEDURE IF EXISTS EmployeeDepartmentData
2
3 DELIMITER $$
4
5 CREATE PROCEDURE EmployeeDepartmentDetails()
6 BEGIN
7     SELECT
8         e.emp_id,
9         e.emp_name,
10        d.dept_name,

```

The 'Result Grid' shows the following data:

emp_id	emp_name	dept_name	salary
101	Rahul	CSE	53000.00
102	Priya	EEE	67000.00
103	Anil	CSE	55000.00
104	Sneha	EEE	70000.00
105	Kiran	ECE	48000.00

The 'Output' window shows the following messages:

#	Time	Action	Message
59	12:16:33	DROP PROCEDURE IF EXISTS DepartmentEmpl...	0 row(s) affected
60	12:16:33	CREATE PROCEDURE DepartmentEmployeeCo...	0 row(s) affected
61	12:16:33	CALL DepartmentEmployeeCount(2)	1 row(s) returned

6. Create a procedure to display departments whose average salary is greater than 55000.

Query 1 x

SQLAddition

Limit to 1000 rows

```
1 DROP PROCEDURE IF EXISTS HighAverageDepartments
2
3 DELIMITER $$
4
5 CREATE PROCEDURE HighAverageDepartments()
6 BEGIN
7     SELECT d.dept_name,
8           AVG(e.salary) AS AverageSalary
9     FROM employees e
10    JOIN departments d
```

Result Grid

	dept_name	AverageSalary
▶	EEE	68500.000000

Result Editor

Result 18 x

Read Only Context Help

Output

Action Output

#	Time	Action	Message
62	12:17:07	DROP PROCEDURE IF EXISTS EmployeeDepart...	0 row(s) affected, 1 warning(s):
63	12:17:07	CREATE PROCEDURE EmployeeDepartmentDet...	0 row(s) affected

7. Create a procedure using IF...ELSE to classify salary as High, Medium or Low.

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS SecondHighestSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE SecondHighestSalary()
6 BEGIN
7     SELECT emp_name, salary
8     FROM (
9         SELECT
10            emp_name,

```

Result Grid

emp_name	salary
Priya	67000.00

Result Grid

Form Editor

Result 19 x Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
65	12:17:39	DROP PROCEDURE IF EXISTS HighAverageDe...	0 row(s) affected, 1 warning(s): 1305 PF
66	12:17:39	CREATE PROCEDURE HighAverageDepartment...	0 row(s) affected
67	12:17:39	CALL HighAverageDepartments()	1 row(s) returned

8. Create a procedure that accepts a department ID and returns its average salary.

Query 1 x

```

1 DROP PROCEDURE IF EXISTS AboveDepartmentAverage
2
3 DELIMITER $$
4
5 CREATE PROCEDURE AboveDepartmentAverage()
6 BEGIN
7     SELECT e.emp_name,
8           e.salary,
9           d.dept_name
10    FROM employees e

```

Result Grid

emp_name	salary	dept_name
Anil	55000.00	CSE
Sneha	70000.00	EEE

Output

#	Time	Action	Message
68	12:18:34	DROP PROCEDURE IF EXISTS SecondHighest...	0 row(s) affected, 1 warning(s):
69	12:18:35	CREATE PROCEDURE SecondHighestSalary() B...	0 row(s) affected
70	12:18:35	CALL SecondHighestSalary()	1 row(s) returned

9. Create a procedure using INOUT to apply a bonus to an input salary.

Query 1 x

```

1 DROP PROCEDURE IF EXISTS ApplyBonus;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE ApplyBonus(
6     INOUT salary_amount DECIMAL(10,2)
7 )
8 BEGIN
9     SET salary_amount = salary_amount + 5000;
10 END $$

```

Result Grid

UpdatedSalary
55000.00

Output

#	Time	Action	Message
73	12:19:14	CALL AboveDepartmentAverage()	2 row(s) returned
74	12:19:56	DROP PROCEDURE IF EXISTS ApplyBonus	0 row(s) affected

10. Create a procedure that performs two related updates inside a transaction.

Query 1 x

```

1 DROP PROCEDURE IF EXISTS TransferSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE TransferSalary(
6     IN fromEmp INT,
7     IN toEmp INT,
8     IN amount DECIMAL(10,2)
9 )
10 BEGIN

```

Result Grid

	emp_id	emp_name	salary
▶	101	Rahul	51000.00
▶	103	Anil	57000.00
•	NULL	NULL	NULL

employees 22 x

Output

Action Output

#	Time	Action	Message
77	12:19:56	CALL ApplyBonus(@salary)	0 row(s) affected
78	12:19:56	SELECT @salary AS UpdatedSalary LIMIT 0, 1000	1 row(s) returned
79	12:20:38	DROP PROCEDURE IF EXISTS TransferSalary	0 row(s) affected, 1 warning
80	12:20:38	CREATE PROCEDURE TransferSalary(IN fro...	0 row(s) affected

7. Common Errors

Procedure already exists: Use DROP PROCEDURE IF EXISTS before recreating it.

Query 1 x

```

1 CREATE PROCEDURE GetEmployees()
2 BEGIN
3     SELECT * FROM employees;
4 END

```

Output

Action Output

#	Time	Action	Message
80	12:20:38	CREATE PROCEDURE TransferSalary(IN fro	0 row(s) affected

Syntax error near BEGIN: Check DELIMITER \$\$ and DELIMITER ;.

Query 1 x

Limit to 1000 rows

```

1 DELIMITER $$
2
3 • CREATE PROCEDURE Example()
4 BEGIN
5     SELECT * FROM employees;
6 END $$
7
8 DELIMITER ;

```

Output

Action Output

#	Time	Action	Message
81	12:20:38	CALL TransferSalary(101,103,2000)	0 row(s) affected
82	12:20:38	SELECT emp_id, emp_name, salary FROM emplo...	2 row(s) returned

Table does not exist: Run USE employeedb1; and SHOW TABLES;

Query 1 x

Limit to 1000 rows

```

1 • SELECT * FROM employees;

```

Result Grid

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	51000.00
	102	Priya	3	67000.00
	103	Anil	1	57000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00
•	NULL	NULL	NULL	NULL

employees 25 x

Output

Action Output

#	Time	Action	Message
---	------	--------	---------

Unknown column: Check names using DESCRIBE employees;